

CSE 575: Advanced Cryptography

Zhengjie Min

September 11, 2026

Abstract

This is a template for TCS (and other kind of) lecture notes.

Contents

I	Information-theoretic Security	2
1	Introduction	3
1.1	A Secret-Writing Example	3
1.1.1	No Secret Given	3
1.1.2	Secret Key	3
2	Shannon / Perfect Secrecy	5
2.1	Definitions	5
2.2	One-time Pad	6
2.3	The Downsides	6
3	Computational Hardness and One-way Functions	8
3.1	Computational Hardness	8
3.2	One-way Functions	8
3.3	Does OWF Exist?	9
A	Algebra	11
B	Analysis	12

Part I

Information-theoretic Security

Chapter 1

Introduction

A classic methodology for a cryptography problem is described as:

1. Form a realistic model of the problem (i.e. what does the “honest” parties want and what “adversary” can do).
2. Define the desired functionality of the system and the security properties against the adversary.
3. Construct and analyze a solution, and ideally prove that it satisfies the definition mentioned in the previous step (under some assumptions).

1.1 A Secret-Writing Example

Following the methodology above, we consider a simple secret-writing problem. Alice wants to send a secret message to Bob over an insecure channel, where Eve can eavesdrop on the communication. The goal is to ensure that Eve cannot learn anything about the message.

1.1.1 No Secret Given

1. Modeling: both Alice and Bob are honest parties. The adversaries are essentially algorithms:
 - Alice is the encoder with algorithm $\text{Enc} : M \rightarrow C$, where M is the finite message space and C is the finite ciphertext space.
 - Bob is the decoder with algorithm $\text{Dec} : C \rightarrow M$.
 - Eve is the eavesdropper with algorithm $E : C \rightarrow ???$.
2. Functionality and Security:
 - Correctness: $\text{Dec}(\text{Enc}(m)) = m$ for all $m \in M$.
 - Security (secrecy of the message): $E(\text{Enc}(m))$ should not reveal any information about m for any adversary E . However, as Bob is honest, no one can stop Eve from setting $E = \text{Dec}$. In this way, $E(\text{Enc}(m)) = m$ from the correctness.

As the first attempt fails, we need to modify the model:

Remark 1.1.1. We can introduce a key to distinguish the honest parties from the adversary.

1.1.2 Secret Key

In Attempt 2, we distinguish the honest parties by introducing a secret key k shared between Alice and Bob. The model is modified as follows:

1. Modeling:

-
- $\text{KeyGen} : \emptyset \rightarrow K$, where K is the finite key space. This is a randomized algorithm that outputs a key $k \in K$.
 - Alice is the encoder with algorithm $\text{Enc} : K \times M \rightarrow C$ (potentially randomized)
 - Bob is the decoder with algorithm $\text{Dec} : K \times C \rightarrow M$.

2. Functionality and Security:

- Correctness: $\text{Dec}(k, \text{Enc}(k, m)) = m$ for all $m \in M$ and $k \in K$. This implies Alice and Bob must share the same key k .
- Security: Eve doesn't know about k , so he has to try every possible key $k' \in K$ to decode the ciphertext. However, different keys may lead to different messages, so Eve cannot be sure about the original message m . We'll formalize it in the next chapter.

Note. The Dec can't be randomized as it needs to give a deterministic output with probability 1.

Chapter 2

Shannon / Perfect Secrecy

2.1 Definitions

Last chapter, we introduced the concept and reason for an SKE.

Definition 2.1.1 (Symmetric Key Encryption (SKE)). A **symmetric key encryption scheme** is a tuple of algorithms (Gen, Enc, Dec) such that:

- Gen is a probabilistic algorithm that outputs a key k from the key space $\mathcal{K}$.
- Enc is a probabilistic algorithm that takes a key k and a message m from the message space $\mathcal{M}$ and outputs a ciphertext c from the ciphertext space $\mathcal{C}$.
- Dec is a deterministic algorithm that takes a key k and a ciphertext c and outputs a message m or an error symbol $\perp$.

The scheme must satisfy the correctness property: for all keys $k \in \mathcal{K}$ and messages $m \in \mathcal{M}$, we have $\text{Dec}(k, \text{Enc}(k, m)) = m$.

For security, there're a lot of ways to define it. We might say the scheme is safe if the adversary cannot recover 90% of the message, or if the adversary can't distinguish c from random noise. Here we adopt Shannon's idea:

Remark 2.1.1. The secrecy of a message means that the ciphertext should reveal nothing about the message that the adversary didn't already know.

In other words, the "posterior" distribution of the message (conditioned on the ciphertext) should be the same as the "prior" distribution of the message. This leads to the following definition:

Definition 2.1.2 (Shannon Secrecy). For any probability distribution D over $\mathcal{M}$, an SKE scheme (Gen, Enc, Dec) is said to achieve **Shannon secrecy** if for every message $\bar{m} \in \mathcal{M}$ and every ciphertext $\bar{c}$ in the range of Enc(k, m) for some $k \leftarrow \text{Gen}$ and $m \leftarrow D$, we have

$$\Pr_{m \leftarrow D, k \leftarrow \text{Gen}}[m = \bar{m} \mid \text{Enc}(k, m) = \bar{c}] = \Pr_{m \leftarrow D}[m = \bar{m}]$$

Now we play with this definition:

$$\begin{aligned} \text{LHS} &= \frac{\Pr_{m,k}[m = \bar{m} \wedge \text{Enc}(k, m) = \bar{c}]}{\Pr_{m,k}[\text{Enc}(k, m) = \bar{c}]} \\ &= \frac{\Pr_m[m = \bar{m}] \cdot \Pr_k[\text{Enc}(k, \bar{m}) = \bar{c}]}{\Pr_{m,k}[\text{Enc}(k, m) = \bar{c}]} \quad (\text{by independence of } m \text{ and } k) \end{aligned}$$

To let $\text{LHS} = \text{RHS}$, we need

$$\Pr_{k \leftarrow \text{Gen}} [\text{Enc}(k, \bar{m}) = \bar{c}] = \Pr_{m \leftarrow D, k \leftarrow \text{Gen}} [\text{Enc}(k, m) = \bar{c}]$$

For a same distribution D , the RHS is a constant. So we can say that for any $\bar{m}$ and $\bar{c}$, the probability of $\bar{c}$ being the ciphertext of $\bar{m}$ is a constant. This means that for any two messages m_0 and m_1 , the probability of $\bar{c}$ being the ciphertext of m_0 is equal to that of m_1 . This leads to a second definition:

Definition 2.1.3 (Perfect Secrecy). An SKE scheme $(\text{Gen}, \text{Enc}, \text{Dec})$ is said to achieve **perfect secrecy** if for every pair of messages $m_0, m_1 \in \mathcal{M}$ and every ciphertext $\bar{c} \in \mathcal{C}$, we have

$$\Pr_{k \leftarrow \text{Gen}} [\text{Enc}(k, m_0) = \bar{c}] = \Pr_{k \leftarrow \text{Gen}} [\text{Enc}(k, m_1) = \bar{c}]$$

In other words, the ciphertext has a same distribution regardless of the message.

Theorem 2.1.1. Shannon secrecy is equivalent to perfect secrecy.

Here we have already proved the $\Rightarrow$ direction. The other direction is left as an exercise.

2.2 One-time Pad

The Shannon / perfect secrecy is achievable.

Construction 2.2.1 (One-time Pad (OTP) / Vernam Cipher). Let $\mathcal{M} = \mathcal{K} = \mathcal{C} = \{0, 1\}^n$ for some n . The one-time pad scheme is defined as follows:

- **Gen:** Choose a key k uniformly at random from $\{0, 1\}^n$.
- **Enc(k, m):** Compute the ciphertext $c = m \oplus k$, where $\oplus$ denotes bitwise XOR.
- **Dec(k, c):** Compute the message $m = c \oplus k$.

Correctness holds as $\forall k, m : \text{Dec}(k, \text{Enc}(k, m)) = (m \oplus k) \oplus k = m$.

Theorem 2.2.1. The OTP scheme achieves perfect secrecy.

Proof. Fix $m_0, m_1, \bar{c} \in \{0, 1\}^n$. We have

$$\Pr_{k \leftarrow \text{Gen}} [\text{Enc}(k, m_0) = \bar{c}] = \Pr_{k \leftarrow \text{Gen}} [m_0 \oplus k = \bar{c}] = \Pr_{k \leftarrow \text{Gen}} [k = m_0 \oplus \bar{c}] = 2^{-n}$$

Symmetrically, the same probability holds for m_1 . Thus, the OTP scheme achieves perfect secrecy. ■

We even have a stronger result: regardless of the message distribution, the ciphertext is uniformly random over the choice of k .

2.3 The Downsides

OTP has some downsides:

- $\mathcal{K} = \mathcal{M}$, which means the key length must be at least as long as the message length. This is impractical for long messages.
- The key can only be used once. Reusing the key for multiple messages compromises security, as

$$\text{Enc}(k, m_0) \oplus \text{Enc}(k, m_1) = (m_0 \oplus k) \oplus (m_1 \oplus k) = m_0 \oplus m_1$$

which reveals information about the messages.

Theorem 2.3.1. In any Shannon / perfect secrecy scheme,

$$|\mathcal{K}| \geq |\mathcal{M}|$$

Proof. Suppose $|\mathcal{K}| < |\mathcal{M}|$.

Take D to be uniform over $\mathcal{M}$. Suppose we get a ciphertext $\bar{c}$ in the range of Enc .

Consider $P = \{\text{Dec}(k, \bar{c}) : k \in \mathcal{K}\}$. Any message that can produce $\bar{c}$ must be in P , by correctness.

However, as $|P| \leq |\mathcal{K}|$ (as each key produces at most one message), we have $|P| < |\mathcal{M}|$. Thus, there exists a message $m^* \in \mathcal{M} \setminus P$.

This means that $\Pr_{m \leftarrow D, k \leftarrow \text{Gen}}[m = m^* \mid \text{Enc}(k, m) = \bar{c}] = 0$, while $\Pr_{m \leftarrow D}[m = m^*] = 1/|\mathcal{M}| > 0$. This contradicts the definition of Shannon secrecy. Therefore, we must have $|\mathcal{K}| \geq |\mathcal{M}|$. ■

Chapter 3

Computational Hardness and One-way Functions

Shannon secrecy is too hard to achieve in practice (a key longer than the message is not desirable). Now we relax those limits by restricting adversary “feasible”, or “efficient” computation.

3.1 Computational Hardness

Here we introduce some conventions and paradigms:

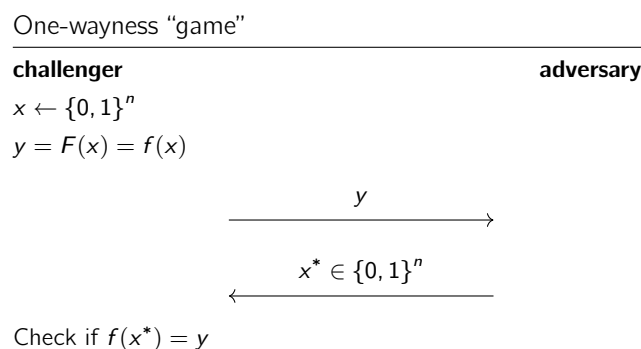
Definition 3.1.1 (Feasible / Efficient Computation). A computation is said to be **feasible** or **efficient** if it can be done in polynomial time, i.e. $O(n^c)$ for a constant c . n is called the **security parameter**.

Definition 3.1.2 (Tiny / Negligible Function). A function $v(n)$ is said to be **tiny** or **negligible**, written as $v(n) = \text{negl}(n)$, if $v(n) = O(1/n^c)$ for every constant c .

For example, $1/2^n$ and $1/n^{\log n}$ are negligible, but $1/n^5$ is not.

3.2 One-way Functions

Intuitively, a one-way function is a function that is easy to compute but hard to invert. To formalize this, we'll do a one-wayness experiment:



Remark 3.2.1. We don't simply check $x^* = x$ because there're collisions. Consider $f(x) = 0, \forall x \in \{0, 1\}^n$. The probability of $x = x^*$ will have probability at most $1/2^n$. It's negligible, but such a function can't be considered secure.

Definition 3.2.1 (One-way Function (OWF)). A function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is called a **one-way function** if:

- Easy to evaluate: there exists a polynomial-time algorithm F that evaluates f : $F(x) = f(x) \forall x \in \{0, 1\}^*$.
- For any probabilistic polynomial-time (PPT) adversary $\mathcal{I}$,

$$\text{Adv}_f^{\text{ow}}(\mathcal{I}) := \Pr_{x \leftarrow \{0, 1\}^n} [\mathcal{I}(1^n, y = f(x)) \in f^{-1}(y)] = \text{negl}(n)$$

Remark 3.2.2. 1^n here gives a guarantee on the input size. Only by 1^n can we be sure that the adversary is poly-time in n instead of something else. It's just a convention and we'll omit such notations in the future.

Nowadays, scientists are working on PRGs and PRFs. They comply secure encryption with $|K| \ll |M|$, many-message security, and so on.

3.3 Does OWF Exist?

The bad news is that the existence of OWFs can indicate $P \neq NP$. As this question remains open by the time we take the course, we don't know about the existence of OWFs.

But the good news is they appear to be abundant in practice. Here we give some examples:

Example 3.3.1 (Subset Sum). $f_{ss} : (\mathbb{Z}_N)^n \times \{0, 1\}^n \rightarrow (\mathbb{Z}_N)^n \times \mathbb{Z}_N$ with $N = 2^n$.

$$f_{ss}(\vec{a}, \vec{b}) = (\vec{a}, s = \sum_{i=1}^n a_i b_i \bmod N) = (\vec{a}, \langle \vec{a}, \vec{b} \rangle \bmod N)$$

The inversion game is as follows. We choose $\vec{a} \in \mathbb{Z}_N^n$, $\vec{b} \leftarrow \{0, 1\}^n$, and give $y = (\vec{a}, s)$ to $\mathcal{I}$. $\mathcal{I}$ outputs $(\vec{a}^*, \vec{b}^*)$ and wins if $f_{ss}(\vec{a}^*, \vec{b}^*) = y \Leftrightarrow \vec{a}^* = \vec{a}$ and $\langle \vec{a}^*, \vec{b}^* \rangle = s \bmod N$.

Appendix

Appendix A

Algebra

Theorem A.0.1. For any polynomial $A \in \mathbb{C}[X]$ of degree $n - 1$, any point set $\{p_0, \dots, p_{n-1}\}$ of size n , $\{(p_i, A(p_i))\}_{i=0}^{n-1}$ determines $A(x)$ and vice versa.

Theorem A.0.2 (Fundamental Theorem of Algebra). Every non-constant single-variable polynomial with complex coefficients has at least one complex root.

In math terms, for $A(x) = \sum_{i=0}^{n-1} a_i x^i$, either $\exists x \in \mathbb{C}$ such that $A(x) = 0$, or $a_0 = a_1 = \dots = a_{n-1} = 0$.

Appendix B

Analysis

Theorem B.0.1 (Master Theorem (simplified)). Let the recurrence relation be:

$$T(n) = aT\left(\frac{n}{b}\right) + n^d$$

where $a \geq 1$ is the number of subproblems, $b > 1$ is the factor by which the problem size is reduced, and $d \geq 0$. The asymptotic bound of $T(n)$ is given by:

1. If $\log_b a > d$, then $T(n) = O(n^{\log_b a})$.
2. If $\log_b a = d$, then $T(n) = O(n^d \log n)$.
3. If $\log_b a < d$, then $T(n) = O(n^d)$.

Theorem B.0.2 (Master Theorem). Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence:

$$T(n) = aT(n/b) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

- **Case 1:** If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
- **Case 3:** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n (Regularity Condition), then $T(n) = \Theta(f(n))$.